Nvidia greenboost: transparently extend GPU VRAM using system RAM/NVMe (gitlab.com/isolatedoctopi)

463 points by mmastrac 4 months ago | hide | past | favorite | 136 comments

0xbadcafebee 4 months ago | next [–]

You can already do this with some GPU drivers:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash amdttm.pages_limit=5242880 ttm.pages_limit=5242880"
```

One downside is your kernel isn't going to reserve that memory away from userland. You will still see all the memory at system level as "free". As the GPU driver starts using it, other apps/the OS will try to use the "free" memory, not knowing how much of it is in use (it may show up as "cache", or not at all). Then OOM killer starts going or programs start crashing, and at some point the OS tips over or GPU driver crashes. You can add loads of swap as a compromise and it works okay, if a bit slow.

In any case, loading a gigantic model just to use system RAM is absurdly slow (due to mem bandwidth), like 1-5 t/s, so it's not practical. It'd take a whole day to process one 86k token request. Just pay a cloud provider $0.01 to do it in 10 seconds.

jmward01 4 months ago | parent | next [–]

The point is not how fast it is now. The point is that this opens new possibilities that can be built on. Potentially models that are trained with slightly different architectures to optimize to this use case. Possibly others come to improve this path. Possibly HW manufacturers make a few small adjustments that remove bottlenecks. Who knows, the next person may combine CPU compute with this mem sharing to get another token a second. Then the next person does predictive loading into memory to keep that bandwith 100% maxed and usable. Then the next does and the next does. Before you know it there is a real thing there that never existed.

This is a great project. I love the possibilities it hints at. Thanks for building it!

smallnamespace 4 months ago | root | parent | next [–]

It’s architecturally not a good approach. System RAM is much slower so you should put data that doesn’t need to be used often on it. That knowledge is at the application layer. Adding a CUDA shim makes system RAM appear like VRAM, which gets things to run, but it will never run very well.

The benchmarks at the bottom mention memory tiering and manually controlling where things go, but if your application already does that, then you probably don’t also need a CUDA shim. The application should control the VRAM to system memory transfers with boring normal code.

jbverschoor 4 months ago | root | parent | next [–]

Not true for unified systems. And for strix halo you need to dedicate the amount which is annoying.

You’re basically stating that swapping is also a bad idea. And to take it further, any memory or storage is a bad idea because there’s L1 cache/SRAM which is faster then the rest

Tuna-Fish 4 months ago | root | parent | next [–]

On some workloads, swapping is a bad idea.

The fundamental problem here is that the workload of LLMs is (vastly simplified) a repeated linear read of all the weights, in order. That is, there is no memory locality in time. There is literally anti-locality; When you read a set of weights, you know you will not need them again until you have processed everything else.

This means that many of the old approaches don't work, because time locality is such a core assumption underlying all of them. The best you can do is really a very large pool of very fast ram.

In the long term, compute is probably going to move towards the memory.

zozbot234 4 months ago | root | parent | next [–]

The main blocker with swapping is not even the limited bandwidth, it's actually the extreme write workload on data elements such as the per-layer model activations - and, to a much lesser extent, the KV-cache. In contrast, there are elements such as inactive experts for highly sparse MoE models, where swapping makes sense since any given expert will probably be unused. You're better off using that VRAM/RAM for something else. So the logic of "reserve VRAM for the highest-value uses, use system RAM as a second tier, finally use storage as a last resort or for read-only data" is still quite valid.

rnrn 4 months ago | root | parent | next [–]

How do get the weights for the right set of experts for a given batch of tokens into fast memory at the right time?

The activated experts is only available after routing, at which point you need the weights immediately and will have very poor performance if they are across PCIe

zozbot234 4 months ago | root | parent | next [–]

Once your model is large enough you'll have to eat the offload cost for *something*, and it might as well be something where most of that VRAM footprint isn't even used. For current models, inactive experts arguably fit that description best. Of course, it may be the case that shifting that part of the graph to CPU compute is a better deal than paying the CPU-to-GPU cost for the active weights and computing on GPU; that's how llama.cpp does it.

dataflow 4 months ago | root | parent | prev | next [–]

> You’re basically stating that swapping is also a bad idea.

Is that a crazy thing to say? I can't recall the last time I was grateful for swap; it might've been before 2010.

dahart 4 months ago | root | parent | next [–]

Try turning swap off and really find out if you’re not grateful for it. Might be fine if you’re never using all your RAM, but if you are, swap off isn’t fun and you might realize you’ve been unconsciously grateful this whole time. ;) Swap might be important for GPU usage even when not using something like greenboost, since display GPUs sometimes use system RAM to back the GPU VRAM.

dataflow 4 months ago | root | parent | next [–]

> Try turning swap off and really find out if you’re not grateful

Er, I did exactly this over a decade ago and never looked back. It's literally one of the first things I do on a new machine.

> Might be fine if you’re never using all your RAM

That's definitely happened occasionally, and no, swap almost always just makes it worse. The thrashing makes the entire machine unusable instead of making the allocating app(s) potentially unstable. I've recovered most times by just immediately killing the app I'm using. And in fact I have warnings that sometimes tell me fast enough before I reach the limit to avoid such issues in the first place.

literalAardvark 4 months ago | root | parent | prev | next [–]

If you've used any unreserved VM ever you're grateful for swapping.

Somewhat indirectly but still.

Tsiklon 4 months ago | root | parent | prev | next [–]

Strix Halo’s unified setup is pretty cool. In systems with 128GB of memory, in BIOS set the dedicated GPU memory to the smallest permitted and the Drivers will use the whole main memory pool appropriately in Linux and Windows

stuaxo 4 months ago | root | parent | next [–]

Does this work on the open source amdgpu drivers ?

I've been a bit too busy to turn mine on for a while.

0xbadcafebee 4 months ago | root | parent | next [–]

The OSS AMDGPU drivers by default allocate a fixed percentage of system RAM for GTT (up to 75%), they do not automatically use the entire system memory. You can override this with the kernel options I posted in my original comment, but as I mentioned, there are some serious negative consequences. You also may need to disable IOMMU or use PT mode. Personally I have had a lot of crashes as a result of this stuff, so I went back to the defaults and just don't run big models.

The biggest factor of whether AMD GPUs on Linux are a PITA or not is ROCm. Strix Halo is supported in ROCm 6.x, so it should be supported on most platforms (I haven't tested it tho). ROCm 7.x is supposed to be better but not all apps support it yet.

AMD, if you're reading this, please hire more SWEs. Nvidia will continue to dominate until you beat them at software.

Tsiklon 4 months ago | root | parent | prev | next [–]

I’ve had no issues running GPT-OSS 120b with decent performance on the machine (HP Zbook Ultra G1a). Running on Bluefin/Universal Blue and Windows.

imtringued 4 months ago | root | parent | prev | next [–]

It's not true for unified systems, because they have no secondary RAM that could be used to extend the GPU memory.

It's pretty weird to insist on a counterargument that has no implications or consequences to the presented argument.

Yes, swapping is a bad idea.

Your second argument also falls flat, because the standard CUDA hardware setup doesn't use CXL so cache coherence isn't available. You're left with manual memory synchronization. Pretending that GPUs have cache for system RAM when they don't is pretty suspect.

timnetworks 4 months ago | root | parent | prev | next [–]

Some people are not concerned with having it run the fastest, just having it run at all may be enough.

m-schuetz 4 months ago | root | parent | next [–]

From my experience, accessing system RAM from the GPU is so slow, it might as well count as "does not work". It's orders of magnitudes faster to memcpy large swaths of memory that you are going to use to the GPU, rather than accessing system mem from a kernel which then takes ages to wait for that small block/page of memory, then waits again for the next small page/block of memory, etc. Latency hiding doesnt work anymore if the latency is that large.

dahart 4 months ago | root | parent | next [–]

You’re right for some workloads, but not all of them. The same could have been said for disk swap since the beginning though, and people still found it valuable. Disk swapping with spinning drives did used to be multiple orders of magnitude slower than RAM. But it prevented applications or the system from crashing.

Using system memory from the GPU isn’t that bad if your compute is high enough and you don’t transfer that much data. There are commercial applications that support it and only see low 2-digit percentage perf impact and not the multiples you might expect. Plus on Windows on Nvidia hardware, the driver will automatically use system memory if you oversubscribe VRAM, and I believe this was introduced to support running Stable Diffusion on smaller GPUs.

nl 4 months ago | root | parent | prev | next [–]

But then you can use CPU/RAM offload, which already allows you to offload without a kernel module.

jmward01 4 months ago | root | parent | prev | next [–]

> It’s architecturally not a good approach.

Yes, with current LLMs and current hardware and current supporting software this is a true statement. My point wasn't that this approach suddenly changes that, it was that it makes it easier to explore alternatives that might change that. Let's imagine some possibilities:

- Models that use a lot of weight reuse: If you strategically reuse layers 3-4x that could give a lot of time for async loading of future weights.

- Models that select experts for several layers at a time: Same thing, while crunching on the current layer you have teed-up future layers that can be transferring in

- HW makers start improving memory bandwidth: This is already happening right? AMD and Apple are pushing unified memory architectures with much higher bandwidth but still not quite there compared to GPUs. This could lead to a hybrid approach that makes those machines much more competitive. similarly, HW makers could bring back technologies that died on the vine that could help, things like Intel's optaine come to mind. Start making mass storage as fast as system memory is now and the equation may change.

These are quick dart throws that probably have obvious holes in them but the point is platforms like this help us explore paths that appeared dead-end until that one change makes them viable and then allows them to take over. It may not happen. It may be a dead end. But that logic means we will never go out on a limb and try something new. We need people and tech that challenges assumptions and makes it easy for people to try out ideas to keep the tech ecosystem evolving. This does that. Even if this particular project doesn't succeed it is a great thing to do if for no other reason it likely just spurred a bunch of people to try their own crazy hacks for LLM inference. Maybe it even enabled a use case with GPUs that nobody realized existed and has nothing to do with LLMs.

adrian_b 4 months ago | parent | prev | next [–]

With discrete GPUs, using system RAM is slow not due to mem bandwidth, but due to PCIe bandwidth, which is the bottleneck.

For example, 16x PCIe 4.0: 256 Gb/s, 16x PCIe 5.0: 512 Gb/s, while 2x DDR5-6400 DIMMs: 819 Gb/s. The actual throughput is lower for both PCIe and DDR5, due to communication overhead.

On server/workstation motherboards which may have 4, 8 or 12 DIMMs instead of 2, the ratio between memory bandwidth and PCIe bandwidth becomes proportionally higher, so the memory throughput achievable by the GPU becomes a very small fraction of the system memory bandwidth.

Tsiklon 4 months ago | root | parent | next [–]

The difference between DDR4 and 5 is quite substantial. I have a fully loaded Cascade Lake Mac Pro - 6 channels of DDR4-2933 gets me to about 120GB/s or 960Gb/s. PCIe 3.0 is a major Achilles heel of what would be a capable workstation system with modern nvidia GPUs precisely for the reason you document.

zozbot234 4 months ago | root | parent | prev | next [–]

> slow not due to mem bandwidth, but due to PCIe bandwidth, which is the bottleneck.

> On server/workstation motherboards ... the memory throughput [to system RAM] achievable by the GPU becomes a very small fraction of the system memory bandwidth.

Yes, this is a critical point. It means that this is only realistically useful for prefill, which is compute- and not memory-bandwidth bound.

shdudns 4 months ago | root | parent | next [–]

Sorry, I'm a bit of a noob on llm. What is "prefill"? As opposed to what?

natechlin 4 months ago | root | parent | next [–]

Prefill - module computes KV cache over input toks, up to the last token in your input (the 'prompt'), at which point it can then begin -

Decode - the model chooses a new token to append to the end of the current token list (i.e. it generates a token), then computes the new tokens KVs.

Decode is basically prefill 1 tok -> add 1 tok -> prefill 1 more tok -> ....

but in the initial prefill stage it doesn't need to do generation, since you've provided the toks.

ghm2199 4 months ago | root | parent | next [–]

And Incidentally prefill would also be how caching,say, a system prompt saves you some $ for API usage with LLM providers. They only compute the kv cache for the new tokens after the system prompt.

Melatonic 4 months ago | root | parent | prev | next [–]

Maybe then this is a forward thinking feature for when we (maybe) get improved GPU hardware slots?

edit: Are you sure PCI-E is even that fast? Looking at the chart on Wikipedia (did not research further - so grain of salt here) shows much lower throughput

lelanthran 4 months ago | parent | prev | next [–]

> any case, loading a gigantic model just to use system RAM is absurdly slow (due to mem bandwidth), like 1-5 t/s, so it's not practical. It'd take a whole day to process one 86k token reques

So don't use it for large requests. Ideal for when you just want to categorise things, for example, "does this task need a shell" or "bucket this email into one of help request, bill due or personal comms".

zozbot234 4 months ago | root | parent | next [–]

The best use is actually for a layer that "almost fits" into VRAM, such that automated offloading to system RAM will be rare enough that it doesn't impact performance.

usrusr 4 months ago | root | parent | next [–]

As in when your secondary memory is fast enough, after the first 10% of the model are processed you can swap their memory with the part for 50% to 60% and when that is done you swap back to have the 0-10% ready in time for the next iteration?

Sounds ambitious, for the small improvement in effective capacity. In particular when I start wondering if real life speed differences would be small enough for that 10% increase, or if it would be even smaller. And that's before factoring in power/cooling cost for saturating another interface.

robotswantdata 4 months ago | parent | prev | next [–]

12 channel ddr5 5600 ECC is around 500gbs which in real world works very well for large MoE

adrian_b 4 months ago | root | parent | next [–]

You mean 500 GB/s, not Gb/s (actually 537 GB/s).

Unfortunately that does not matter. Even in a cheap desktop motherboard the memory bandwidth is higher than of 16-lane PCIe 5.0.

Therefore the memory bandwidth available to a discrete GPU is determined by its PCIe slot, not by the system memory.

If you install multiple GPUs, in many MBs that will halve the bandwidth of the PCIe slots, for an even lower memory throughput.

zargon 4 months ago | root | parent | next [–]

> in many MBs that will halve the bandwidth of the PCIe slots

Not on boards that have 12 channels of DDR5.

But yeah, squeezing an LLM from RAM through the PCIe bus is silly. I would expect it would be faster to just run a portion of the model on the CPU in llama.cop fashion.

Gracana 4 months ago | root | parent | next [–]

It is much faster, yeah. llama.cpp supports swapping between system memory and GPU, but it's recommended that you don't use that feature because it's rarely the right call vs using the CPU to do inference on the model parts in system CPU memory.

Edit: the settings is "GGML_CUDA_ENABLE_UNIFIED_MEMORY=1"... useful if you have unified memory, very slow if you do not.

lostmsu 3 months ago | root | parent | prev | next [–]

llama.cpp afaik does not run portion of the model on the CPU. --cpu-moe just offloads weights to RAM, but they are still loaded to GPU for compute.

robotswantdata 4 months ago | root | parent | prev | next [–]

Talking about dual socket SP5 EPYC with 24 DIMM slots, 128 PCIe 5.0 lanes

It's fast for hybrid inference, if you get the KV and MoE layers tuned between the Blackwell card(s) and offloading.

We have a prototype unit and it's very fast with large MoEs

RobotToaster 4 months ago | parent | prev | next [–]

Would MoE models work better with this approach?

rnrn 4 months ago | prev | next [–]

Why is there a new kernel driver here at all? It appears that all it does it allocate system RAM ("DDR4") and export it as a dmabuf for import to cuda as mapped external memory. Then a userspace shim hijacks APIs to use that if gpu memory is full. cuda already supports allocating mapped system memory, so AFAICT this could be implemented in the userspace shim with no new kernel driver.

Also as other commenters have mentioned, redirecting allocations to managed memory would also enable similar oversubscription

And the hijack approach only makes sense for making apps have this behavior with no changes, and could be done with minor app changes (e.g. PyTorch has a pluggable allocator interface). App changes also enable intentionally placing specific allocations.

My impression is that this is vibe from beginning to end, starting from a design that only makes sense if you are hallucinating

Melatonic 4 months ago | parent | next [–]

Maybe theres a significant latency advantage to doing it this way?

Or, as you said, making everything backwards compatible that is not being regularly updated

daneel_w 4 months ago | prev | next [–]

Related, a couple of years ago: https://old.reddit.com/r/Amd/comments/15t0lsm/i_turned_a_95_...

"I turned a $95 AMD APU into a 16GB VRAM GPU and it can run stable diffusion!"

3abiton 4 months ago | parent | next [–]

> it can generate a 50 steps 512x512 image around 1 minute and 50 seconds.

I have the 4650G APU, and the best way to describe it is: lacking of support. This was even more true 3 yo than now. rocm (is) was absolutely dogshit then, I know this because I tried to do the same when that post was made. You have to compile everything from scratch, get the relevant patches, and even then, xformers which is a library that accelerate diffusion model inferencing was not supported for renoir or rocm back then. Yes, you can generate an image, but it was much slower, and rigged with bugs. You couldn'rt update rocm because it broke compatibility, and it was partly the reason I got into nixos. That being said, those APUs are a power house. Nowadays I can run decent agentic workflows on them (I have 64gb of ddr4 ram, ie APU can suck as much as it needs with the latest linux kernels).

Just note, diffusion models are still second class citizens on AMD apus even GPUs. But then again, nothing close right now on the market except for what apple offers.

nl 4 months ago | root | parent | next [–]

The Ryzen AI CPU/GPUs (Ryzan AI 395+ etc) seem to have increasing support - https://lemonade-server.ai/ now has support for the NPU as well as the combined CPU/GPU (which I guess is a APU but is different to the G series of APUs I think?)

But I'm always interested in first hand experiences of how good is it *really* - I'm pretty cynical about the idea that AMD actually knows what it takes to build good software end-to-end.

3abiton 4 months ago | root | parent | next [–]

I also have one, and indeed support is very much frictionless now compared to a year ago. But again, not thanks to AMD, as initially it was purely community driven. Strix halo was not even supported by ROCm (officially), and we had to deal with therock images, then donato made the toolbox, and then lemonade came through. I am really surprised how AMD approached this. They made big promises, they threw the hardware out, it really is amazing piece of hardware given what you can do with it, but it was left hanging

without support for AI stack for months even though it had it in its name. Contrast that with the DGX spark (yes it had and still had bugs in its kernels, but cuda worked on day 1) and you can see the difference. Nvidia is selling an ecosystem, AMD is selling hardware. I really hope AMD focus on the software layer more.

nl 4 months ago | root | parent | next [–]

I believe Lemonade *is* the AMD team right?

But yes I agree with you about their lack of prioritization for software!

zozbot234 4 months ago | root | parent | prev | next [–]

Note that Lemonade Server uses NPU low-level code that's proprietary, not available as open source. It would be nice to work on a fully open alternative, perhaps by exposing the NPU itself as a Vulkan Compute-capable device, that shaders can be auto-compiled to.

nl 4 months ago | prev | next [–]

This is really interesting engineering, but I agree with the other commentators that the benchmarking makes it hard to understand the contribution various factors are having.

The *ExLlamaV3 EXL3 2bpw (8 GB, full VRAM)* row is an order of magnitude faster than the baseline - but the baseline seems to be the 32GB model running with the KV cache shared to system memory only (I think?)

But if a 8GB model gives sufficient quality then it seems like that would have worked without the shared memory thing?

I *think* the useful apples-to-apples benchmark is currently the *Ollama + GreenBoost shim (baseline)* (2-5 tps) vs *ExLlamaV3 + GreenBoost cache* (8–20 tps) comparison.

It would be really useful to see this compared with the existing llama CPU/memory offload. There is a note at the start ("Offload layers to CPU — works, but drops token/s by 5–10× because CPU RAM has no CUDA coherence") - but it is unclear if that 5-10x token speed drop is compared to running a model completely in GPU or compared to the greenboost approach.

I think it is vs GPU, in which case it seems likely the performance is similar to what greenboost is giving but probably much more stable.

kristianp 4 months ago | parent | next [–]

ExLlamaV3 EXL3 2bpw is likely the 30b parameter GLM 4.7 Flash quantised down to 2 bits, the unstated assumption is that you need to check the 2bpw quantisation works well enough for your use case.

The reported size of the ModelOpt FP8, 16 GB, sounds wrong to me. If its 8 bits per parameter it is going to be a similar size to the glm-4.7-flash:q8_0. They repeat this a few times in the readme.

yjtpesesu2 4 months ago | prev | next [–]

How does this differ from anything llama.cpp offers, regarding offloading layers? The repo consistently refers to "DDR4". Is there a reason DDR5 won't work with this?

svnt 4 months ago | parent | next [–]

The readme opens with this:

> I have an RTX 5070 with 12 GB VRAM and I wanted to run glm-4.7-flash:q8_0, which is a 31.8 GB model. The standard options are:

> Offload layers to CPU — works, but drops token/s by 5–10× because CPU RAM has no CUDA coherence. You end up waiting. Use a smaller quantization — you lose quality. At q4_0 the model is noticeably worse on reasoning tasks.

> Buy a bigger GPU — not realistic for consumer hardware. A 48 GB card costs more than a complete workstation.

> None of those felt right, so I built an alternative: route the overflow memory to DDR4 via DMA-BUF, which gives the GPU direct access to system RAM over PCIe 4.0 without a CPU copy involved.

And then limps home with this caveat on the closest thing to a benchmark:

> The PCIe 4.0 link (~32 GB/s) is the bottleneck when the model overflows VRAM. The best strategy is to shrink the model until it fits — either with EXL3 quantization or ModelOpt PTQ — and use GreenBoost's DDR4 pool for KV cache only.

I think the reason it refers it to DDR4 is because that is how the user explained it to their coding agent. LLMs are great at perpetuating unnecessary specificity.

moffkalast 4 months ago | root | parent | next [–]

Given that 32 GB/s is significantly worse than CPU to RAM speeds these days, does the additional compute really make it any faster in practice? The KV cache is always on the GPU anyway unless you're doing something really weird, so it won't affect ingestion, and generation is typically bandwidth bound. With something like ×16 PCIe 6.0 it would actually make sense, but nothing less than that, or maybe for smaller dense models that are more compute bound with 8x PCIe 6.0 or 16x 5.0 but that's already below DDR5 speeds.

zozbot234 4 months ago | root | parent | next [–]

Additional compute is generally a win for prefill, while memory bandwidth is king for decode. KV cache however is the main blocker for long context, so it should be offloaded to system RAM and even to NVMe

swap as context grows. Yes that's slow on an absolute basis but it's faster (and more power efficient, which makes everything *else* faster) than not having the cache at all, so it's still a huge win.

moffkalast 4 months ago | root | parent | next [–]

Well if you do that then you reverse the strengths of your system. It might be best to work with the context length you can offload, like a normal person.

itsTyrion 3 months ago | root | parent | prev | next [–]

> "wanted to run glm-4.7-flash:q8_0" > q8_0

a well made (as in, unsloth) smaller quant will help a good amount here, without a notable reduction in performance or increase in perplexity

kcb 4 months ago | parent | prev | next [–]

CUDA has had managed memory that pages between VRAM and system RAM for a decade. Problem is doing so is unusably slow for AI purposes. Seems like an unnecessary layer here.

segmondy 4 months ago | parent | prev | next [–]

I was wondering the same, but llama.cpp was written to offload to system ram. If this really works, then the advantage could be that one could run transformers / sglang, etc or other tools that don't offload to system ram. However, I want to see the numbers. Perhaps I'll give this a try, but I need a throw away box I could trash if something goes wrong, but have none at the moment.

xienze 4 months ago | parent | prev | next [–]

Presumably it means that software doesn’t have to write the same sort of layer offloading support. It’ll “just work” as if you had X GB of VRAM all along.

yjtpesesu2 4 months ago | root | parent | next [–]

so, magic?

Havoc 4 months ago | prev | next [–]

> The best strategy is to shrink the model until it fits — either with EXL3 quantization or ModelOpt PTQ — and use GreenBoost's DDR4 pool for KV cache only.

Does this make sense? I'd have thought the KV is guaranteed to be used 100% of the time while say in a MoE the same can't be said of the weights.

Though I suppose if you're shooting for huge context then having that allocation go into ram makes sense specially when its allocated but not used yet

alexeldeib 4 months ago | parent | next [–]

KV cache is, well, a cache that can fill up and trigger eviction. You require enough space to execute at least 1 fwd pass of 1 request at your context length. KV cache hits reduce TTFT by avoiding prefill. You don’t get to skip decode.

MoE is kinda related in terms of lower usage requirements vs a dense model of same total param size, but I think your mental model is a bit off.

zozbot234 4 months ago | root | parent | next [–]

KV cache is also eminently swappable if you have fast storage, since it mostly sees small append-only writes per token - it's not rewritten continuously like the activations. (I believe it's even better if you use cached input tokens across requests, since that portion of KV cache can then be recycled and save a single ~KV-cache sized write per request.) Accessing swapped-out cache may be slow, but it's highly preferable to not having that cache amount at all and recomputing from scratch.

ma2kx 4 months ago | prev | next [–]

The physical bottleneck to system memory remains. Therefore, I assume that better results are achieved by manually adjusting which layers are offloaded.

I would prefer to use system memory to cache different models, focusing on things like embedding, rerankers, and TTS. This is sufficient to run a more complex RAG locally, for example, via Mem0, and then use a larger LLM via the cloud.

aruametello 4 months ago | prev | next [–]

Post traumatic "nvidia TurboCache" disorder triggered.

https://en.wikipedia.org/wiki/TurboCache

(Not the same thing 1:1, but worth the joke anyway)

yjftsjthsd-h 4 months ago | prev | next [–]

Previously: https://news.ycombinator.com/item?id=47384557

(Still cool, still would benefit from better benchmarks)

armada651 4 months ago | prev | next [–]

Doesn't Windows already do this by default? I can already run models bigger than my GPU VRAM and it will start using up to 50% of my system RAM as "shared memory". This is on a Desktop PC without a shared memory architecture.

nickjj 4 months ago | parent | next [–]

Yep I had a GeForce 750 Ti (2 GB) and I was able to run a ton of things on Windows without any issues at all.

As soon as I switched to Linux I had all sorts of problems on Wayland where as soon as that 2 GB was reached, apps would segfault or act in their own unique ways (opening empty windows) when no GPU memory was available to allocate.

Turns out this is a problem with NVIDIA on Wayland. On X, NVIDIA's drivers act more like Windows. AMD's Linux drivers act more like Windows out of the box on both Wayland and X. System memory gets used when VRAM is full. I know this because I got tired of being unable to use my system after opening 3 browser tabs and a few terminals on Wayland so I bought an AMD RX 480 with 8 GB on eBay. You could say my cost of running Linux on the desktop was $80 + shipping.

A few months ago I wrote a long post going over some of these details at https://nickjanetakis.com/blog/gpu-memory-allocation-bugs-wi.... It even includes videos showing what it's like opening apps both on Wayland and X with that NVIDIA card.

Yokohiii 4 months ago | parent | prev | next [–]

The nvidia windows driver enables RAM swapping by default.

Great way to backstab you if you prefer inference speed.

3836293648 4 months ago | parent | prev | next [–]

I don't think Windows does this, but Ollama does

whywhywhywhy 4 months ago | root | parent | next [–]

It's the drivers but it was a relatively recent addition, think it was added when either the 30xx or 40xx series shipped and the lower cards had pitiful VRAM so they enabled it by default so they'd work with all games.

Most people who know it does this turns it off because it kicks in too early so if you have 24GB it'll offload to RAM and tank your inference speed when you hit around 22GB use.

https://nvidia.custhelp.com/app/answers/detail/a_id/5490/~/s...

lastdong 4 months ago | root | parent | next [–]

Nicely linked!

nodja 4 months ago | root | parent | prev | next [–]

NVIDIA's GPU drivers on windows 100% do this

https://i.imgur.com/c0a3vUy.png

dahart 4 months ago | root | parent | prev | next [–]

The Nvidia driver has used system memory fallback for a couple of years now.

https://nvidia.custhelp.com/app/answers/detail/a_id/5490/~/s...

Insanity 4 months ago | prev | next [–]

Extend your VRAM using RAM, then extend your RAM using Swap.

system2 4 months ago | parent | next [–]

And burn the swap pagesys file to a rewritable DVD to complete the cycle. It will be super fast that way.

SV_BubbleTime 4 months ago | parent | prev | next [–]

If you are doing video models, this is an excellent way to murder your SSD.

Do not put swap on an SSD you care about at all.

rvz 4 months ago | root | parent | next [–]

> Do not put swap on an SSD you care about at all.

This.

Many people rediscovering what the purpose of swap files are, but will still find a way to abuse it without knowing that they are actually destroying their SSD.

zozbot234 4 months ago | root | parent | prev | next [–]

You can of course monitor SMART wearout indicators to check whether this is happening. Casual use of swap for non LLM-use is actually fine since "cold" ephemeral data will be swapped out first and that will never get written to; KV cache is mostly fine since it's similarly append-only so writes are tolerably small; but yes, more general LLM inference totally breaks that limited-writes pattern and will wear out/kill your media.

Insanity 4 months ago | root | parent | prev | next [–]

I was writing it somewhat tongue-in-cheek and not as a serious suggestion. But thanks for adding the disclaimer, that's good advice!

duskdozer 4 months ago | root | parent | prev | next [–]

zram swap otoh should be relatively 'free'

zozbot234 4 months ago | root | parent | next [–]

LLM working memory is not compressible so ZRAM doesn't buy you anything.

krige 4 months ago | parent | prev | next [–]

Extend your RAM using RAM Doubler!

FooBarWidget 4 months ago | root | parent | next [–]

Then extend your disk space using DoubleSpace/DriveSpace!

Datagenerator 4 months ago | root | parent | next [–]

Just to be sure install Stacker (from STAC electronics) too

lossyalgo 4 months ago | root | parent | prev | next [–]

I did that. It worked, until it didn't, and then I learned how to format my 340MB HDD and re-install DOS 6.22. Fun times!

paultendo 4 months ago | prev | next [–]

Could be a very useful way to do some overnight tasks using spare RAM. Possibly things like LLM-based categorisation, labelling, data cleansing. That's what comes to mind for me anyway.

MaxikCZ 4 months ago | parent | next [–]

Neat part is every task becomes overnight task when you start offloading to RAM.

152334H 4 months ago | prev | next [–]

Nobody mentioning how this project is vibecoded slop?

```
  > The code is really bad with completely uneeded parts. The LLM (Qwen 2.5 7B) has hardcoded the i9 14700KF
topology, and has variables related to it never used... It's even funnier that the show hardware function always
prints the same string. There are even random pip log files. Why did this slop got coverage here?
```

https://www.phoronix.com/forums/forum/linux-graphics-x-org-d...

dwroberts 4 months ago | prev | next [–]

The title here needs changing, this is for nvidia cards but it is not an official project and has nothing to do with them

(Feels especially deceptive when there is another top story right with the headline “nvidia nemoclaw” which *is* an official project)

ninjagoo 4 months ago | prev | next [–]

This is awesome! Normally, offloading layers to the CPU RAM means that the compute for those layers occurs on the CPU instead of the GPU, generally speaking. The CPU is orders of magnitude slower than the GPU.

With this approach the compute occurs on the GPU, with the tradeoff that layers in RAM have to be moved back-and-forth through PCI-DMA. It seems to me that this should offer a speedup vs compute split between GPU and CPU. The amount of speedup will depend on how many layers would have been on CPU compute, minus the reduction due to moving those layers between RAM and the GPU.

What's slower? Compute on the CPU or moving data from RAM to GPU through PCI-DMA?

wewewedxfgdf 4 months ago | prev | next [–]

Why don't they just put ram slots on the card so you can augment the fast ram

M95D 4 months ago | parent | next [–]

Speed and reliability. A connector of any kind reduces signal quality. Data lines need to be longer, because the memory slot won't fit under the radiator where the memory chips are now, and that adds even more electrical interference and degrades signal.

Also, we had memory slots on '90s cards. They were extremely expensive and proprietary. Ever saw a Matrox VRAM card? I never did.

whalesalad 4 months ago | root | parent | next [–]

I am hoping that we seriously evolve the ATX standard to allow for a socketed GPU board that can also enable user replaceable memory. Seeing an enormous GPU that is larger than the motherboard itself hanging from a PCI slot feels like horse and buggy shit. I'm imaging two boards back-to-back connected by a central high

bandwidth bus (which could also do power delivery) that would allow one side of the case to be for CPU/RAM and the other side to be for GPU/VRAM.

M95D 4 months ago | root | parent | next [–]

Your solution only allows for one GPU, maybe two if the motherboard is really huge, and it doesn't really solve the slotted VRAM problem.

PCI is (was) allowed to be even longer. Old AT and ATX cases had a slotted support bracket to hold the far end of the PCI cards. See how an Adaptec 2400A looks like.

Gracana 4 months ago | root | parent | prev | next [–]

SOCAMM2 could work. Nvidia's using it on the Vera Rubin boards, as seen here: https://www.pchardwarepro.com/wp-content/uploads/2025/11/que...

HighGoldstein 4 months ago | root | parent | prev | next [–]

> A connector of any kind reduces signal quality.

Like the M.2 connector?

> Data lines need to be longer

Like the data lines going all the way to an on-motherboard storage device?

literalAardvark 4 months ago | root | parent | next [–]

Soldered stuff is still dramatically better than the M2 connector (than any connector really). You've never wondered why RAM doesn't use PCI Express?

zbentley 4 months ago | root | parent | prev | next [–]

> Like the M.2 connector?

Yes, though likely something with a higher pin count since memory access is more likely to be random and can be parallel versus block storage.

> Like the data lines going all the way to an on-motherboard storage device?

Yes. Why would a GPU manufacturer/packager take on that cost, if it’s presently served well enough for most people by offloading it onto other parts of the system?

adrian_b 4 months ago | root | parent | prev | next [–]

The current DIMM and SODIMM modules cannot be used for much higher speeds than are available now.

This is why there are several proposals of improved forms for memory modules, which use different sockets, like LPCAMM2, which should be able to work with faster memories.

However even LPCAMM2 is unlikely to work at the speeds of soldered GDDR7.

varispeed 4 months ago | root | parent | next [–]

Can't they make it easier to solder / desolder?

adrian_b 4 months ago | root | parent | next [–]

It is not very difficult to solder/desolder, but you need suitable tools, which are not cheap.

Moreover, when you do this manually, unless it is something that you do every day it may be quite difficult to be certain that soldering has been done well enough to remain reliable during long term use. In the industry, very expensive equipment is used to check the quality of soldering, e.g. X-ray machines.

So unlike inserting a memory module in a socket, which is reasonably foolproof, soldering devices is not something that could be used in a product sold to the general population.

When I was young, there still existed computer kits, where you soldered yourself all the ICs on the motherboard, so you could get a computer at a much lower price than for a fully assembled computer. My first PC was of this kind.

However, at that time PCs were still something that was bought by a small fraction of the population, which were people that you could expect to be willing to learn things like how to solder and who would be willing to accept the risk of damaging the product that they have bought. Today PCs are addressed to the general public, so nobody would offer GPU cards that you must solder.

zargon 4 months ago | root | parent | prev | next [–]

Yes and yes. NVMe storage is very slow, so it can get away with such things.

VHRanger 4 months ago | parent | prev | next [–]

GDDR7x doesn't come in dimm factor?

In general soldered ram seems to get much higher bandwidth than removeable ram. See ryzen AI Max vs 9950x max ram throughputfor example

nic547 4 months ago | root | parent | next [–]

Strix Halo uses a 256bit memory interface, the normal desktop processors only have a 128bit interface, that's the biggest difference in bandwidth. For more bandwidth you need to go to a Threadripper.

Strix Halo seems to use LPDDR with 8000 MT/s, which is a bit faster than the usual 5600 MT/s-6400 MT/s "normal" DDR5-DIMMs (Albeit (expensive) faster ones seem to exist), so there's a slight edge towards soldered memory (not sure about LPCAMM2 and similar tech).

GDDR7 is a different league, a 5070 Ti also has a 256bit memory interface, but has 896GB/s bandwidth, compared to strix halo with 256GB/s

VHRanger 4 months ago | root | parent | next [–]

It's really hard to push DDR5 past 6000MT/s on 4+ DIMMs it seems.

I had to get everything top spec to fit 4 channels of 6000MT/s on my 9950x (asus proArt motherboard and the top tier trident neo RAM sticks) -- otherwise it's reportedly unstable.

nic547 4 months ago | root | parent | next [–]

9950X is dual channel, running 4 DIMMs runs them interleaved, with two DIMMs sharing one physical connection, impacting signal integrity severely. AFAIK this has gotten worse with DDR5 to the point that it's generally recommended to avoid 4 DIMMs unless you really can't get enough RAM otherwise. For maximum bandwidth you need to avoid interleaving.

Strix Halo simply has more memory controllers. Threadrippers are also quad channel, and should be able to run 4 DIMMs at rated speeds, but the cheapest Zen 5 Threadripper seems to be almost double the price of a 9950X3D.

VHRanger 4 months ago | root | parent | next [–]

And I presume that doubled price is before you look at the workstation class motherboards, which also tend to be much more expensive.

Thanks for the info on the hardware quirks, useful to know!

We seem to be arriving at a cambrian explosion of viable hardware these days between ARM and x86, soldered vs DIMM, etc.

It's refreshing coming from 20 years of x86 being all that matters.

adrian_b 4 months ago | root | parent | prev | next [–]

No.

All GDDR memory is intended only for being soldered around a GPU chip, on the same PCB. This is how they achieve a memory throughput that is 4 to 8 times higher than the DDR memories used in DIMMs or SODIMMs.

wewewedxfgdf 4 months ago | root | parent | prev | next [–]

We are talking here about slower ram to augment.

timmmmmmay 4 months ago | parent | prev | next [–]

connectors are bad for signal integrity and GDDR is particularly picky about this

wewewedxfgdf 4 months ago | root | parent | next [–]

We're talking about ordinary RAM to augment, like a cache.

Not as GPU VRAM expansion.

bhewes 4 months ago | prev | next [–]

This has been fun we can task our nemotron-3-super model to run over night when our desktops are idle. 4070s and 96gb of ram works fine. Slow but does it's job.

sabareesh 4 months ago | prev | next [–]

I wish it provided benchmark comparing Direct RAM offload vs CPU offload vs Full VRAM

bguberfain 4 months ago | prev | next [–]

"A watchdog kernel thread monitors RAM and NVMe pressure and signals userspace before things get dangerous." - which kind of danger this type of solution can have?

felipe_aramburu 4 months ago | prev | next [–]

How does this relate to cuCascade https://github.com/nvidia/cucascade

tandr 4 months ago | prev | next [–]

Some simpler benchmark table would be great. May I suggest Ollama on base machine, Ollama with T1, Ollama with T1+T2 etc. on midsize and big models to compare token/sec?

yalogin 4 months ago | prev | next [–]

Is there a use case for this today? Feels more like nvidia is priming the software hoping system designers will find ways to use it.

dr_kretyn 4 months ago | prev | next [–]

Is there a similar initiative for AMD?

angry_octet 4 months ago | prev | next [–]

I have a system with an ungodly amount of Optane memory and I'm hoping this will work.

Rafuino 4 months ago | parent | next [–]

What do you have? I've got a 905P and a 900P and am already using these in LM Studio by putting all models there and extending system memory with more scratch space... Not sure if I need to do anything differently with this since LM Studio already enabled it I think

angry_octet 4 months ago | root | parent | next [–]

The DIMM version, PMEM 200 modules. Appears as /dev/dax and you can mmap it. Could also use as a virtual filesystem, or using RAM as a cache for pmem, but sceptical of that.

Is the virtual memory system is a good way to expand memory for inference, via having it directly manage a non uniform pool? I haven't seen any research on that.

Berazu 4 months ago | prev | next [–]

I wish there was a way to extend RAM/NVMe with GPU VRAM. :(

nuopnu 4 months ago | parent | next [–]

There are vram disks, so at least you can use it for the swap.

pabs3 4 months ago | prev | next [–]

Would be great to get this into mainline Linux.

bandrami 4 months ago | prev | next [–]

Qu'ils mangent de la brioche

brador 4 months ago | prev | next [–]

Could this work on steam deck?

NooneAtAll3 4 months ago | prev | next [–]

nvidia failed to provide gpu with actually meaningful amount of vram

and instead of improving the actual product, it decided to "solve the problem in software"

I expect this greenboost to fall and burn, honestly...

cma 4 months ago | parent | next [–]

> it decided to "solve the problem in software"

This isn't made by nvidia

shmeeed 4 months ago | root | parent | next [–]

Still kinda true, though. As other commenters have pointed out, their Windows drivers do similar stuff.

holoduke 4 months ago | prev [–]

The is extremely slow and not useful in my opinion.

daneel_w 4 months ago | parent | next [–]

It makes the difference between being able to run a lot of machine learning tasks, and not being able at all. Pretty useful.

majorchord 4 months ago | parent | prev | next [–]

I would say it depends entirely on your usecase. I don't think there can be a simple "not useful" generalization that applies to everyone.

jauntywundrkind 4 months ago | root | parent | next [–]

Man I wish that was a canned response that could be deployed on demand! Well said.

I really appreciate thriftful & resourceful points of view. Exploring what if, looking for use is such a great virtue.

bigwheels 4 months ago | parent | prev | next [–]

Can you elaborate beyond the shallow/superficial dismissal?

whywhywhywhy 4 months ago | root | parent | next [–]

If it takes seconds in VRAM it can take tens of minutes running the same thing offloaded to RAM if it hasn't been designed to do it.

ozgrakkurt 4 months ago | parent | prev [–]

It is about as useful as rtx